

1. Introducción a los informes

- 1.1 Qué es un informe y para qué se utiliza
- 1.2 Informes incrustados vs externos (embedded / non-embedded)
- 1.3 Casos de uso reales en aplicaciones Java

2. Herramientas actuales de generación de informes

- 2.1 JasperReports Library
- 2.2 **Jaspersoft Studio (Eclipse-based)**
- 2.3 Alternativas (BIRT, Reporting Services – solo contextual)

3. Jaspersoft Studio

- 3.1 Instalación y requisitos
- 3.2 Interfaz de usuario de Jaspersoft Studio
- 3.3 Estructura de un informe (.jrxml)
- 3.4 Bandas y elementos del informe

4. Orígenes de datos

- 4.1 JDBC (MySQL / PostgreSQL / MariaDB)
- 4.2 JavaBeans y colecciones
- 4.3 CSV, XML y JSON
- 4.4 Uso de Data Adapters

5. Creación de informes. Primeros pasos.

6. Gestión de errores

7. Formatos de salida.

8. Definir parámetros de entrada en Jaspersoft Studio

9. Imágenes

10. Gráficas

11. Agrupamientos y subinformes

1. Introducción a los informes

En el desarrollo de aplicaciones informáticas, especialmente en entornos profesionales y empresariales, la generación de informes constituye una funcionalidad esencial. Aunque las aplicaciones modernas disponen de interfaces gráficas cada vez más ricas, interactivas y dinámicas, estas interfaces no sustituyen la necesidad de **documentos estructurados, persistentes y exportables** que permitan representar información de forma clara, ordenada y comprensible.

Un informe no es simplemente una “vista bonita” de los datos. Se trata de un **producto final** de la aplicación, pensado para ser **consultado, impreso, almacenado, compartido o auditado**, y que debe mantener su validez independientemente de la aplicación que lo generó. Por este motivo, los sistemas de informes se diseñan como un componente separado de la interfaz de usuario y de la lógica de negocio.

Desde un punto de vista arquitectónico, los informes permiten aplicar correctamente el principio de **separación de responsabilidades**:

- La base de datos se encarga de almacenar la información.
- La lógica de negocio procesa y valida los datos.
- El motor de informes se encarga exclusivamente de **presentar la información**.

Esta separación es especialmente relevante en aplicaciones Java, donde los motores de informes como JasperReports permiten desacoplar completamente el diseño del documento del código que lo genera

1.1 Qué es un informe y para qué se utiliza

Un **informe** es un documento generado automáticamente a partir de una fuente de datos estructurada, al que se le aplica un diseño previamente definido con el objetivo de mostrar información de forma clara, coherente y significativa para el usuario final.

Desde el punto de vista técnico, un informe se basa en la combinación de tres elementos fundamentales:

1. **El diseño del informe**, que define cómo se va a presentar la información. Este diseño especifica la estructura del documento, la disposición de los datos, los estilos tipográficos, los encabezados, los pies de página, los cálculos y los elementos gráficos.
2. **La fuente de datos**, que proporciona los valores reales que se mostrarán en el informe. Esta fuente puede ser una base de datos relacional, una colección de objetos Java, un fichero estructurado o incluso el resultado de un servicio externo.
3. **El motor de generación**, que interpreta el diseño, recupera los datos, evalúa expresiones y produce el documento final en el formato deseado.

A diferencia de una pantalla de la aplicación, un informe:

- No depende del estado interactivo del usuario.
- No se modifica una vez generado.
- Tiene un formato fijo y controlado.
- Puede exportarse a distintos formatos estándar.

La finalidad principal de un informe es **convertir datos en información útil**. Mientras que una base de datos almacena grandes volúmenes de datos en bruto, el informe selecciona, organiza, resume y presenta solo aquello que resulta relevante para un determinado propósito.

Entre los usos más habituales de los informes se encuentran:

- La generación de facturas, albaranes y tickets de venta.
- La elaboración de listados de clientes, productos o empleados.
- El análisis de ventas, ingresos o consumos a lo largo del tiempo.
- La obtención de estadísticas y resúmenes para la toma de decisiones.
- El cumplimiento de obligaciones legales y administrativas.

Un aspecto clave de los informes es que permiten **realizar cálculos sobre los datos**, como sumas, medias, porcentajes o totales acumulados, sin necesidad de que estos cálculos estén previamente almacenados en la base de datos. Esto aporta una gran flexibilidad y reduce la redundancia de información.

Además, los informes permiten aplicar formatos visuales que facilitan la interpretación de los datos, como tablas, agrupaciones, gráficos o resaltados condicionales, algo que sería mucho más complejo de implementar directamente en la interfaz de usuario.

1.2 Informes incrustados vs externos (embedded / non-embedded)

A la hora de integrar informes en una aplicación Java, una de las decisiones de diseño más importantes es determinar **dónde se almacenará el diseño del informe y cómo se accederá a él**. En función de esta decisión, podemos distinguir entre informes incrustados (embedded) e informes externos (non-embedded).

Informes incrustados

Un informe incrustado es aquel cuyo diseño forma parte del propio proyecto de la aplicación. El fichero que define el informe se almacena dentro de la estructura del proyecto y se empaqueta junto con el resto de recursos cuando se genera el artefacto final (por ejemplo, un JAR o un WAR).

En este enfoque, el informe se considera un recurso más de la aplicación, al mismo nivel que las imágenes, los estilos o los ficheros de configuración. El acceso al informe se realiza internamente, sin depender de rutas externas ni de recursos remotos.

Este tipo de integración presenta varias ventajas. En primer lugar, garantiza que el informe siempre será compatible con la versión de la aplicación que lo utiliza, ya que ambos evolucionan de forma conjunta. Además, simplifica el despliegue, puesto que no es necesario copiar ficheros adicionales ni configurar rutas específicas en el entorno de ejecución.

Sin embargo, los informes incrustados también tienen limitaciones. Cualquier cambio en el diseño del informe implica recompilar y redistribuir la aplicación, lo que puede resultar poco práctico en entornos donde se requieren modificaciones frecuentes o personalizaciones para distintos clientes.

Por este motivo, los informes incrustados son especialmente adecuados para aplicaciones cerradas, con un número limitado de informes bien definidos y con pocos cambios a lo largo del tiempo.

Informes externos

En el caso de los informes externos, el diseño del informe se almacena fuera del proyecto de la aplicación. El informe se carga en tiempo de ejecución desde una ubicación externa, como el sistema de ficheros, un servidor de informes o un repositorio compartido.

Este enfoque aporta una gran flexibilidad, ya que permite modificar el diseño del informe sin necesidad de recompilar ni redistribuir la aplicación. Es posible, por ejemplo, corregir un formato, añadir un campo o cambiar un encabezado simplemente sustituyendo el fichero del informe.

No obstante, esta flexibilidad tiene un coste. La aplicación debe gestionar correctamente la localización del informe, controlar las versiones y garantizar la compatibilidad entre el diseño del informe y la versión del motor de generación utilizada. Además, se incrementa la complejidad del despliegue y del mantenimiento del sistema.

Los informes externos son habituales en sistemas empresariales complejos, donde:

- Existen muchos informes diferentes.
- Se requieren personalizaciones por cliente.
- El diseño de informes lo realizan perfiles no desarrolladores.
- Se necesita modificar informes en producción con rapidez.

Comparación conceptual

Desde un punto de vista conceptual, la elección entre informes incrustados y externos no es una cuestión de cuál es mejor, sino de **qué solución se adapta mejor al contexto del proyecto**. En aplicaciones pequeñas o educativas, los informes incrustados suelen ser la opción más sencilla y segura. En sistemas grandes y configurables, los informes externos ofrecen una flexibilidad imprescindible.

1.3 Casos de uso reales en aplicaciones Java

La generación de informes es una funcionalidad transversal que aparece en prácticamente cualquier tipo de aplicación Java que gestione información estructurada.

En aplicaciones de escritorio desarrolladas con Swing o JavaFX, los informes suelen utilizarse para generar documentos que el usuario necesita imprimir o guardar, como tickets, facturas, justificantes o listados. En estos casos, el informe actúa como un complemento a la interfaz gráfica, proporcionando una salida formal y persistente de los datos que el usuario maneja en pantalla.

En aplicaciones empresariales, los informes adquieren aún más relevancia. Son habituales los informes de ventas, de rendimiento, de inventario o de auditoría, que permiten analizar grandes volúmenes de datos de forma agregada. Estos informes suelen incluir agrupaciones, subtotales, parámetros y gráficos, y se generan de forma periódica o bajo demanda.

En sistemas de gestión como ERP o CRM, los informes forman parte del flujo normal de trabajo. Cada operación importante genera uno o varios documentos asociados, que deben cumplir formatos estrictos y ser coherentes con la información almacenada en la base de datos. En este contexto, la fiabilidad y la exactitud de los informes es crítica.

En arquitecturas modernas basadas en servicios, los informes pueden generarse en el backend y ofrecerse al frontend como un archivo descargable, normalmente en formato PDF. En estos casos, el informe no se visualiza directamente dentro de la aplicación, sino que se entrega como un recurso independiente, reforzando aún más su carácter de documento autónomo.

Finalmente, en el ámbito educativo y formativo, el trabajo con informes permite al alumnado enfrentarse a problemas reales del desarrollo de software, combinando conocimientos de bases de datos, programación Java, diseño de documentos y arquitectura de aplicaciones. El uso de motores de informes constituye una excelente herramienta para comprender cómo se integran distintos componentes dentro de una aplicación completa.

2. Herramienta de generación de informes

La generación de informes no es una tarea trivial dentro de una aplicación. Requiere combinar **diseño visual**, **acceso a datos**, **evaluación de expresiones**, **cálculos**, **formateo avanzado** y **exportación a múltiples formatos**. Por este motivo, en entornos profesionales no se implementan informes “a mano”, sino que se utilizan **motores de informes especializados**.

Un motor de informes proporciona una infraestructura completa para:

- Definir el diseño del documento de forma declarativa.
- Conectarse a múltiples tipos de orígenes de datos.
- Ejecutar consultas y cálculos.
- Generar documentos finales en formatos estándar.
- Integrarse con aplicaciones existentes.

En el ecosistema Java, **JasperReports** se ha consolidado como una de las soluciones más extendidas y maduras, apoyada por herramientas de diseño como **Jaspersoft Studio**.

2.1 JasperReports Library

Visión general

JasperReports Library es un **motor de generación de informes open source escrito en Java**. No es una aplicación visual ni una herramienta de usuario final, sino una **librería** que se integra directamente en aplicaciones Java para ejecutar informes en tiempo de ejecución.

Su función principal es **tomar un diseño de informe**, combinarlo con una fuente de datos y producir un documento final en uno o varios formatos de salida.

Desde el punto de vista arquitectónico, JasperReports actúa como una **capa intermedia** entre los datos y el documento final, permitiendo desacoplar completamente la lógica de presentación del resto de la aplicación.

Arquitectura y funcionamiento interno

El funcionamiento de JasperReports se basa en varias fases bien definidas:

1. Diseño del informe

El diseño se define en un fichero `.jrxml`, que es un documento XML donde se describen:

- Bandas del informe (title, detail, footer, etc.)
- Campos asociados a los datos
- Parámetros de entrada, variables, estilos y formatos.

2. **Compilación**

El fichero `.jrxml` se compila a un fichero `.jasper`.

En esta fase:

- Se valida la estructura del informe.
- Se generan clases Java internas.
- Se optimiza el informe para su ejecución.

3. **Relleno (fill)**

El informe compilado se rellena con datos reales:

- Se ejecutan consultas SQL o se recorren colecciones.
- Se evalúan expresiones.
- Se calculan variables y totales.

4. El resultado es un objeto `JasperPrint`, que representa el informe completo en memoria.

5. **Exportación o visualización**

A partir del objeto `JasperPrint`, el informe puede:

- Visualizarse en pantalla.
- Exportarse a PDF, HTML, Excel, CSV, etc.
- Enviarse a impresora.

Orígenes de datos soportados

JasperReports es especialmente potente por su capacidad para trabajar con múltiples fuentes de datos, en-tre ellas:

- Bases de datos relacionales mediante JDBC.
- Colecciones de objetos Java (JavaBeans).
- Ficheros CSV.
- Documentos XML.
- Ficheros JSON.
- Consultas HQL (Hibernate).
- Fuentes de datos personalizadas implementadas por el desarrollador.

Esta flexibilidad permite utilizar JasperReports tanto en aplicaciones sencillas como en sistemas empresariales complejos.

Integración en aplicaciones Java

Al tratarse de una librería, JasperReports se integra directamente en el código Java.

Desde el punto de vista del desarrollador, el uso de JasperReports implica:

- Cargar el diseño del informe.
- Pasar los parámetros necesarios.
- Proporcionar una fuente de datos.
- Ejecutar el informe.

Todo este proceso se realiza sin que el usuario final tenga conocimiento del funcionamiento interno del motor, lo que refuerza la separación entre lógica y presentación.

Ventajas y limitaciones

Entre las principales ventajas de JasperReports destacan:

- Madurez y estabilidad.
- Amplia documentación y comunidad.
- Soporte para formatos de salida profesionales.
- Integración directa con Java.

Como limitación principal, cabe señalar que **no incluye por sí mismo un diseñador visual**, lo que obliga a utilizar una herramienta adicional para crear los ficheros `.jrxml`. Esta necesidad es precisamente la que cubre Jaspersoft Studio.

2.2 Jaspersoft Studio (Eclipse-based)

Qué es Jaspersoft Studio

Jaspersoft Studio es un **entorno gráfico de diseño de informes**, basado en la plataforma Eclipse, que actúa como complemento natural de JasperReports Library.

Su objetivo es permitir al desarrollador o diseñador:

- Crear informes visualmente.
- Definir consultas y parámetros sin escribir código.
- Diseñar documentos complejos de forma intuitiva.
- Previsualizar los informes antes de integrarlos en la aplicación.

Jaspersoft Studio sustituye a herramientas antiguas como iReport, que ya no se mantienen.

Características principales

Jaspersoft Studio ofrece un conjunto muy completo de funcionalidades:

- Editor visual WYSIWYG para el diseño del informe.
- Editor de código para modificar directamente el `.jrxml`.
- Gestión de orígenes de datos mediante *Data Adapters*.
- Asistentes para:
 - Informes simples.
 - Subinformes.
 - Gráficos.
 - Tablas y crosstabs.
- Vista previa integrada.
- Soporte para estilos y plantillas reutilizables.

Al estar basado en Eclipse, hereda muchas de sus características, como la organización por proyectos, la gestión de recursos y la posibilidad de personalizar la interfaz.

Flujo de trabajo típico con Jaspersoft Studio

1. Crear un proyecto de informes.
2. Definir uno o varios *Data Adapters* para conectar con las fuentes de datos.
3. Crear un nuevo informe mediante un asistente o desde cero.
4. Diseñar visualmente el informe arrastrando campos y elementos.
5. Definir parámetros, variables y expresiones.
6. Previsualizar el informe con datos reales.
7. Exportar el fichero `.jrxml` o `.jasper` para integrarlo en la aplicación Java.

Este flujo permite detectar errores de diseño o de datos **antes** de integrar el informe.

Rol de Jaspersoft Studio en el desarrollo

Es importante entender que Jaspersoft Studio:

- **No sustituye** a JasperReports Library.
- **No genera informes por sí solo en producción.**
- Se utiliza exclusivamente en la fase de **diseño y prueba**.

La ejecución real del informe en producción siempre la realiza la librería JasperReports desde la aplicación Java.

2.3 Alternativas

Aunque JasperReports es una solución ampliamente utilizada, existen otras

Eclipse BIRT

BIRT (Business Intelligence and Reporting Tools) es un proyecto open source de la fundación Eclipse, orientado principalmente a aplicaciones Java y web. Cuenta con un diseñador visual integrado en Eclipse, motor de informes y buen soporte para gráficos.

Sin embargo, su uso ha disminuido en los últimos años, y su integración con aplicaciones Java de escritorio es menos habitual que la de JasperReports.

Microsoft SQL Server Reporting Services (SSRS)

Reporting Services es la solución de generación de informes del ecosistema Microsoft.

Aunque conceptualmente similar a JasperReports, no es una opción habitual en proyectos Java.

Reflexión final sobre las herramientas

En el contexto de aplicaciones Java y en entornos educativos como DAM, **JasperReports Library junto con Jaspersoft Studio** constituye una solución completa, profesional y alineada con el mundo real.

3. Jaspersoft Studio

Jaspersoft Studio es la **herramienta oficial de diseño de informes para JasperReports** y constituye una pieza clave dentro del flujo de trabajo profesional. Como hemos visto en el apartado anterior, mientras que JasperReports Library es el motor encargado de ejecutar los informes dentro de la aplicación Java, Jaspersoft Studio se utiliza **exclusivamente en la fase de diseño y validación**.

Su objetivo es **permitir crear informes complejos sin necesidad de escribir el código XML del informe**, proporcionando una interfaz visual potente que abstrae gran parte de la complejidad técnica, sin perder control sobre los detalles cuando es necesario.

3.1 Instalación y requisitos

Requisitos previos

Jaspersoft Studio es una aplicación de escritorio desarrollada en Java y basada en la plataforma Eclipse RCP. Para su correcto funcionamiento es necesario disponer de **Java Development Kit (JDK)**.

No es necesario disponer de Eclipse instalado previamente, ya que Jaspersoft Studio se distribuye como una aplicación independiente que incluye todo lo necesario para su ejecución.

Instalación

El proceso de instalación es sencillo:

1. Descargar la versión Community de Jaspersoft Studio desde el sitio oficial.

<https://community.jaspersoft.com/download-jaspersoft/community-edition>

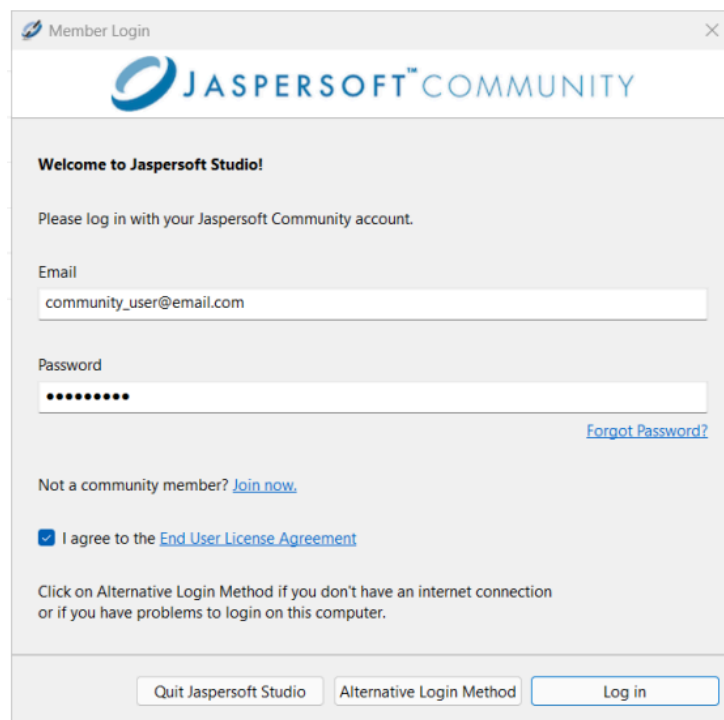
2. Descomprimir el paquete descargado en una ubicación adecuada.
3. Ejecutar el lanzador de la aplicación.

No requiere instalación adicional ni configuración compleja. A diferencia de herramientas antiguas, no es necesario instalar plugins externos ni añadir librerías manualmente para el diseño de informes.

Logging in to the Jaspersoft Community

Cuando se te solicite, en la pantalla de **Inicio de sesión de Miembro**, introduce el **correo electrónico** y la **contraseña**, que debemos obtener previamente:

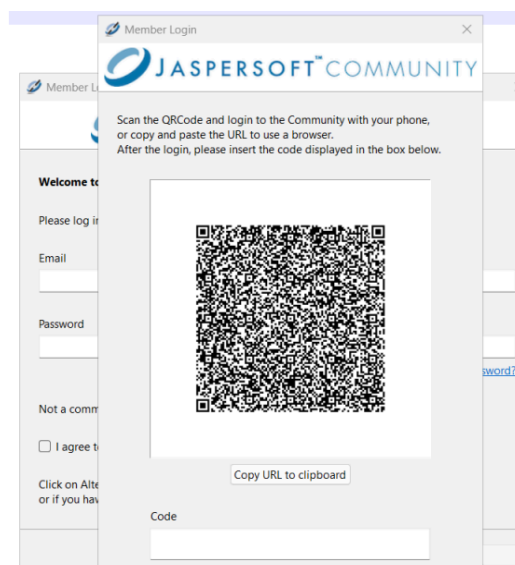
<https://community.jaspersoft.com/register>



The screenshot shows a 'Member Login' dialog box for the Jaspersoft Community. It features the Jaspersoft logo and the text 'Welcome to Jaspersoft Studio!'. Below this, it asks the user to log in with their account. There are input fields for 'Email' (containing 'community_user@email.com') and 'Password' (masked with dots). A 'Forgot Password?' link is located to the right of the password field. Below the password field, there is a checkbox labeled 'I agree to the End User License Agreement' which is checked. At the bottom, there are three buttons: 'Quit Jaspersoft Studio', 'Alternative Login Method', and 'Log in'. A note at the bottom states: 'Click on Alternative Login Method if you don't have an internet connection or if you have problems to login on this computer.'

Alternative Login Method

Permite activar Jaspersoft Studio usando un smartphone cuando no hay o hay poca conexión a Internet. El programa genera un código QR que se escanea para obtener un código de activación.



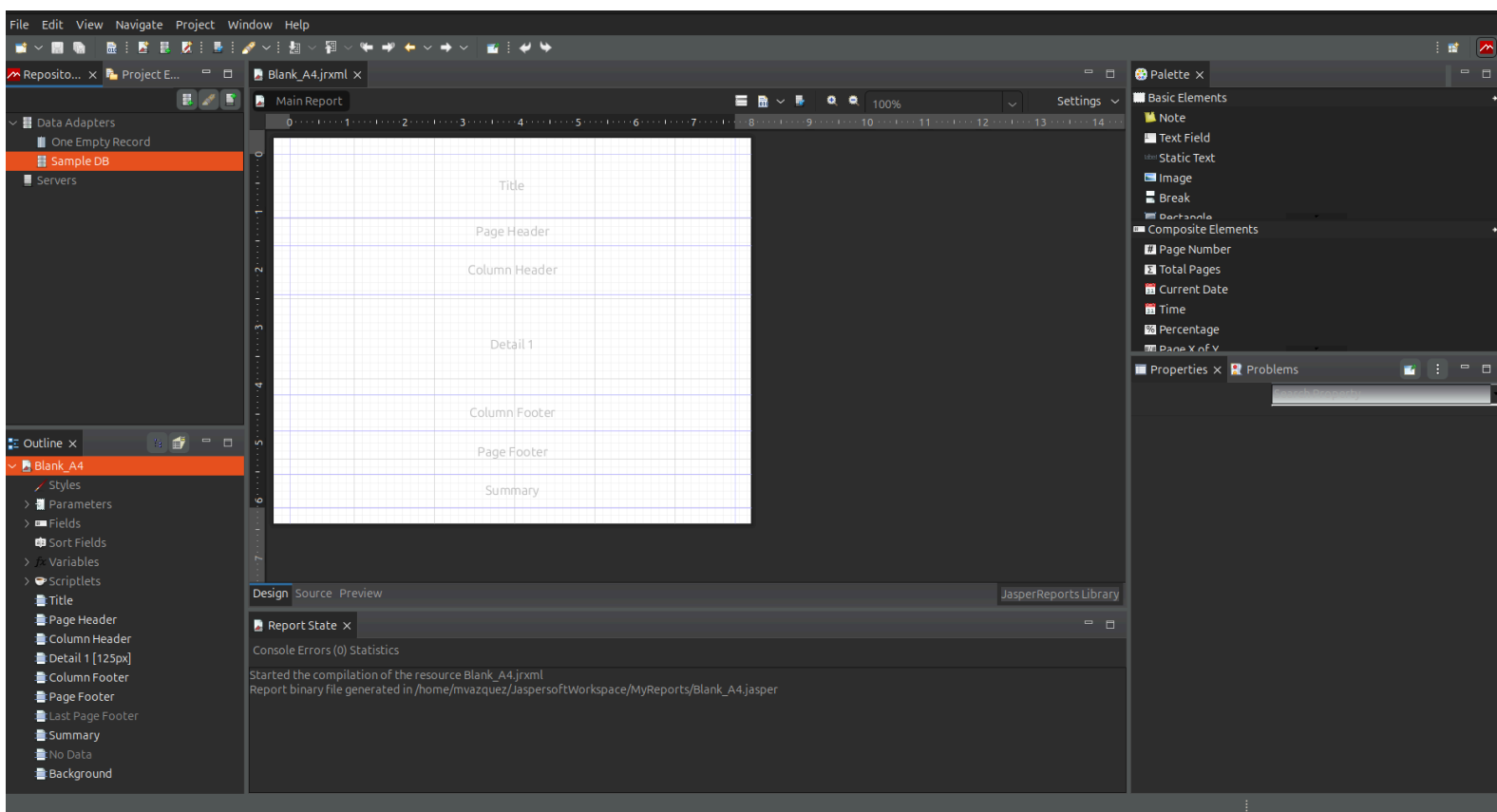
This screenshot shows the 'Alternative Login Method' dialog box. It displays a large QR code in the center. Above the QR code, it instructs the user to 'Scan the QRCode and login to the Community with your phone, or copy and paste the URL to use a browser. After the login, please insert the code displayed in the box below.' Below the QR code is a button labeled 'Copy URL to clipboard'. At the bottom, there is a 'Code' label followed by an empty input field. The background shows a blurred version of the standard login dialog.

Entorno de trabajo

Jaspersoft Studio crea un **workspace**, concepto heredado de Eclipse, que es el espacio donde se almacenan:

- Los proyectos de informes.
- Los diseños **.jrxml**.
- Los adaptadores de datos.
- Los recursos asociados (imágenes, subinformes, estilos).

Este enfoque facilita la organización del trabajo y la reutilización de informes en distintos proyectos.



3.2 Interfaz de usuario de Jaspersoft Studio

La interfaz de usuario de **Jaspersoft Studio** está basada en la plataforma **Eclipse RCP**, lo que implica que adopta un modelo de trabajo organizado en **vistas (views)** y **editores (editors)**. Este enfoque es habitual en entornos de desarrollo profesionales y permite trabajar de forma estructurada con proyectos complejos, aunque al principio pueda resultar visualmente denso.

La interfaz está diseñada para ofrecer **un control muy preciso sobre el diseño del informe**, separando claramente:

- La **estructura del proyecto**
- El **diseño visual del informe**
- La **estructura lógica interna**
- Las **propiedades y validación**
- La **previsualización del resultado**

Por defecto, Jaspersoft Studio presenta una disposición de ventanas pensada para cubrir todo el ciclo de diseño de un informe sin necesidad de cambiar constantemente de contexto.

Estructura general de la interfaz

En su configuración por defecto, la interfaz se organiza en las siguientes zonas principales:

- **Zona izquierda:** exploradores (Repository Explorer / Project Explorer)
- **Zona central:** editor del informe
- **Zona derecha:** paleta de elementos y propiedades
- **Zona inferior:** problemas, salida y mensajes
- **Panel adicional:** Outline (estructura del informe)

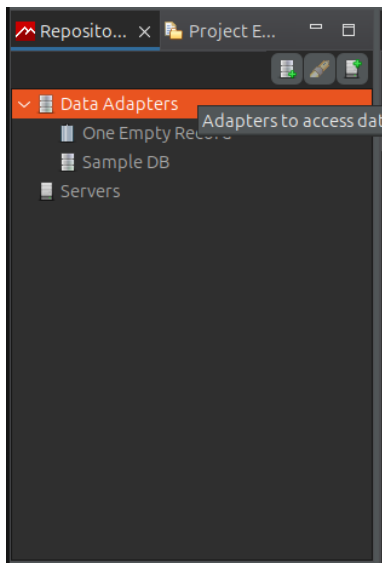
Cada una de estas zonas cumple una función concreta dentro del proceso de diseño.

Repository Explorer / Project Explorer (Workspace)

En el lateral izquierdo de Jaspersoft Studio se encuentra el **explorador del workspace**, que puede presentarse en dos vistas relacionadas pero con responsabilidades ligeramente distintas: **Project Explorer** y **Repository Explorer**.

Este panel constituye el **punto de partida de cualquier proyecto de informes**, ya que centraliza la gestión de **recursos, conexiones y estructuras** necesarias para el diseño y ejecución de los informes.

Project Explorer: estructura del proyecto de informes



El **Project Explorer** muestra la **estructura jerárquica de los proyectos locales** de Jaspersoft Studio, de forma muy similar a un IDE de desarrollo como Eclipse o IntelliJ.

Desde esta vista se organizan los recursos propios del proyecto:

- **Informes principales (.jrxml)**, que representan los documentos finales que se ejecutarán desde la aplicación Java.
- **Subinformes**, utilizados como componentes reutilizables dentro de otros informes.
- **Imágenes**, como logotipos corporativos, iconos o recursos gráficos.
- **Estilos y plantillas**, que permiten unificar la apariencia de varios informes.
- **Otros recursos compartidos**, como ficheros auxiliares o scripts.

Esta organización permite trabajar de forma **modular**, separando claramente:

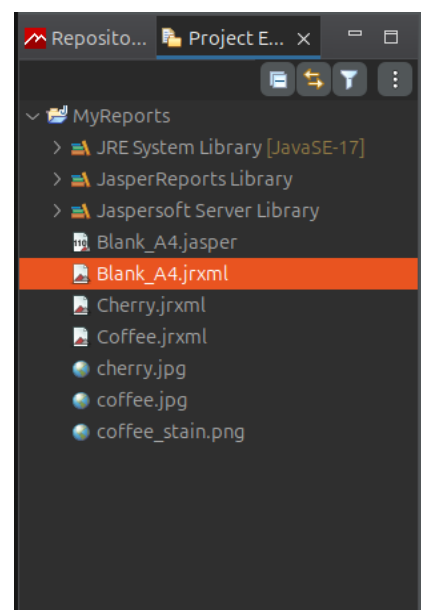
- Informes principales.
- Informes auxiliares o de apoyo.
- Recursos reutilizables comunes a varios informes.

Repository Explorer: repositorio de recursos y conexiones

El **Repository Explorer** tiene un papel distinto y especialmente importante: actúa como un **repositorio central de recursos reutilizables**, siendo el lugar donde se gestionan las **conexiones a bases de datos y orígenes de datos**.

Desde el Repository Explorer se definen y almacenan los llamados **Data Adapters**, que son los objetos que describen **cómo Jaspersoft Studio se conecta a una fuente de datos concreta**, destacan:

- **Data Adapters JDBC**, para conectar con bases de datos relacionales (MySQL, PostgreSQL, MariaDB, etc.).
- **Adaptadores para CSV, XML o JSON**.



Editor de informes (zona central)

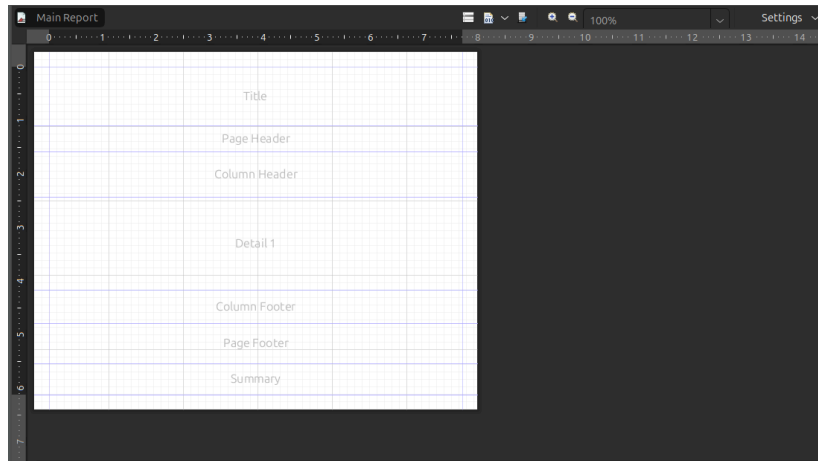
La zona central de la interfaz está ocupada por el **editor del informe**, que es el área principal de trabajo. Aquí se abre el informe seleccionado y se realiza la mayor parte del diseño.

El editor permite alternar entre **tres modos principales**, que representan distintas vistas del mismo informe:

Design (Diseño)

Es la vista visual del informe. En este modo:

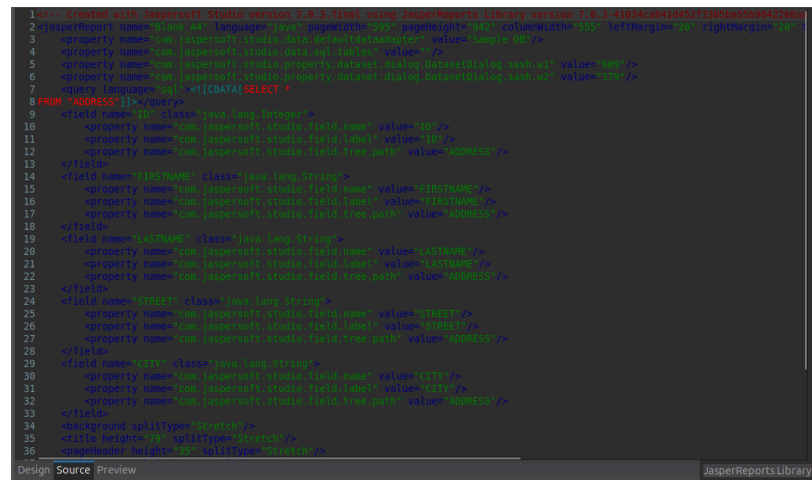
- Se muestran las **bandas** del informe de forma gráfica.
- Se arrastran y colocan los elementos visuales.
- Se ajustan posiciones, tamaños y alineaciones.



Esta vista es la más utilizada durante el diseño inicial del informe, ya que permite trabajar de forma intuitiva y visual, similar a un editor de maquetación.

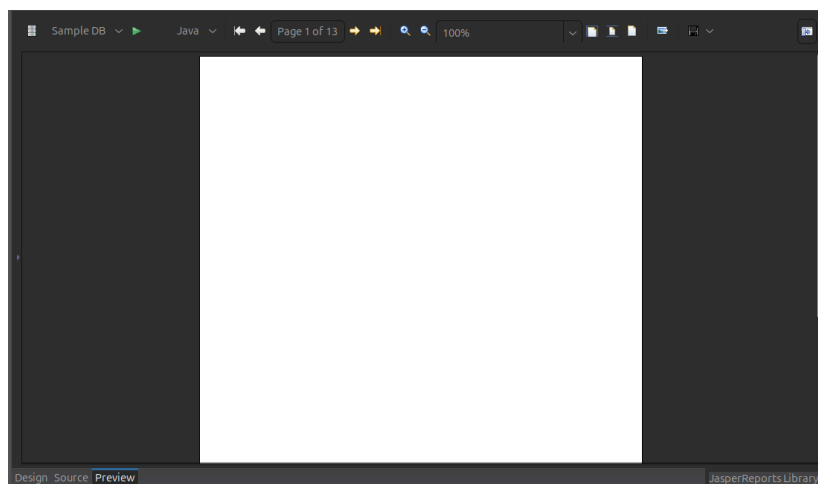
Source (Código)

En esta vista se muestra el **código XML** del fichero **.jrxml** que define el informe. Aunque no es habitual modificar el informe directamente desde aquí en un nivel inicial.



Preview (Vista previa)

La vista *Preview* permite **ejecutar el informe insertándose datos** y visualizar el resultado final directamente desde Jaspersoft Studio.



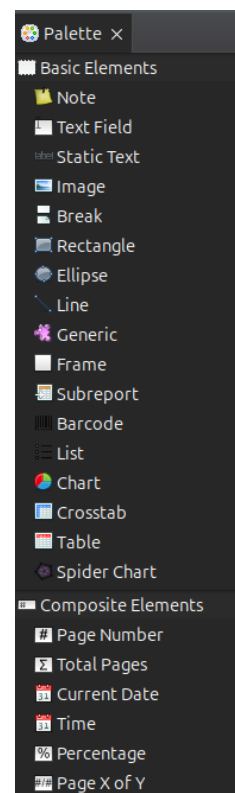
Palette (Paleta de elementos)

Contiene todos los componentes visuales que pueden añadirse a un informe. Entre los elementos más habituales se encuentran:

- Campos de texto dinámicos (*Text Field*)
- Texto estático
- Imágenes
- Líneas y rectángulos
- Tablas
- Gráficos

El uso es directo: los elementos se **arrastran desde la paleta hasta una banda del informe**. Una vez colocados, se configuran mediante la vista de propiedades.

La paleta actúa como el “catálogo” de piezas con las que se construye el informe.



Properties / Problems (Propiedades y problemas)

En la parte derecha o inferior suele encontrarse la **vista de propiedades**, acompañada habitualmente por la vista de **problemas**.

Vista de propiedades

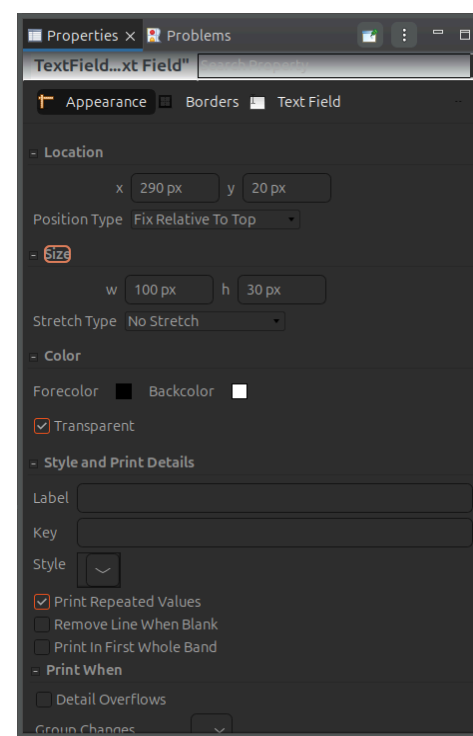
Cada elemento del informe dispone de un conjunto amplio de propiedades que permiten controlar con gran precisión su comportamiento.

Desde esta vista se pueden configurar aspectos como:

- Posición y tamaño exactos
- Tipografía y estilos
- Formato de datos (números, fechas)
- Expresiones asociadas

Vista de problemas

La vista de problemas muestra advertencias o errores de diseño.

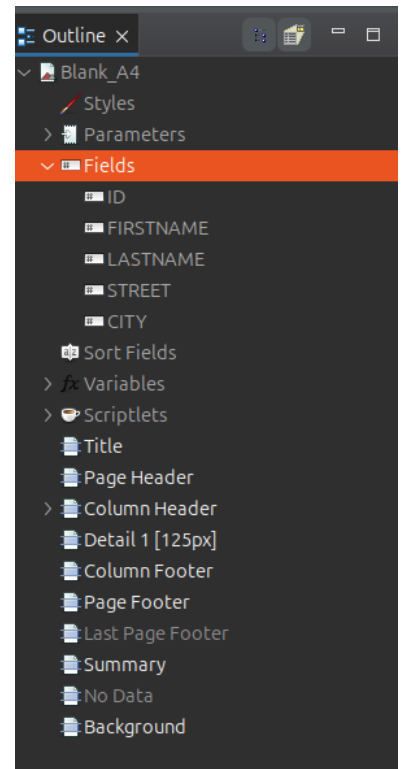


Outline (Estructura del informe)

El panel **Outline** ofrece una vista de la **estructura jerárquica del informe activo**.

Desde aquí se puede visualizar, gestionar y seleccionar:

- Campos/datos de entrada.
- Bandas
- Elementos contenidos en cada banda
- Grupos
- Subinformes
- ...



3.3 Estructura de un informe (.jrxml)

Los informes diseñados en Jaspersoft Studio se almacenan en ficheros con extensión **.jrxml**, que son documentos XML con una estructura bien definida.

Desde un punto de vista conceptual, el **.jrxml** describe **qué debe mostrarse y cómo**, pero no contiene datos reales. Es un modelo declarativo que será interpretado posteriormente por JasperReports.

Aunque el desarrollador no suele escribir este XML manualmente, es importante entender su estructura para:

- Diagnosticar errores.
- Realizar ajustes avanzados.
- Comprender cómo se ejecuta el informe.

Relación con JasperReports Library

Antes de ejecutarse, el fichero **.jrxml** debe ser **compilado**, generando un fichero **.jasper**. Este fichero compilado es el que se utiliza en la aplicación Java para generar el informe final.

Este proceso de compilación:

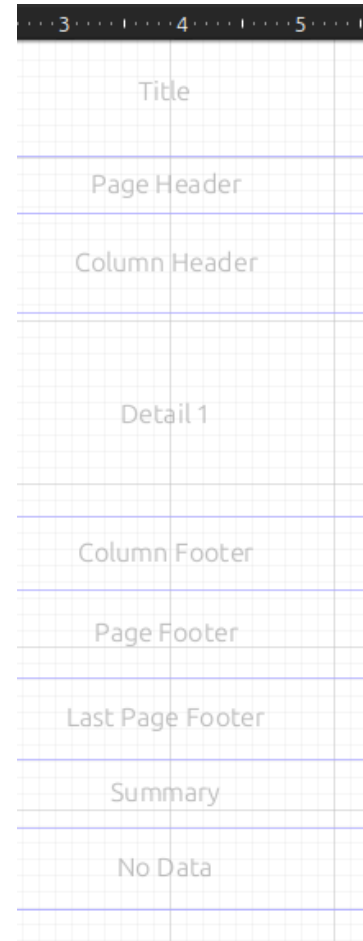
- Valida el diseño.
- Genera clases Java internas.
- Optimiza el informe para su ejecución.

3.4 Bandas y elementos del informe

El concepto de **bandas** es uno de los pilares fundamentales de JasperReports y Jaspersoft Studio. Un informe se organiza verticalmente en secciones denominadas bandas, cada una con un propósito específico.

Bandas principales

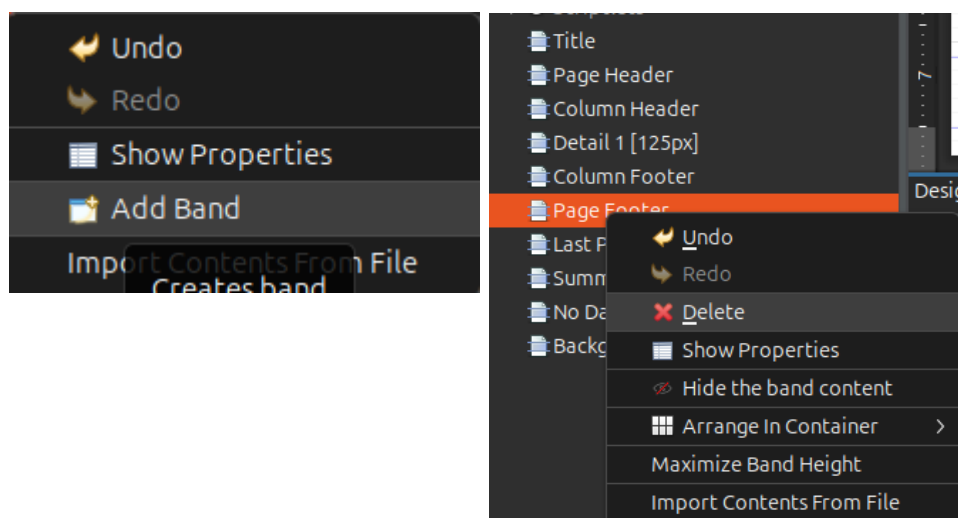
- **Title:** Se muestra una única vez, al comienzo del informe. En este espacio pondremos el título del informe, logotipo...
- **Page Header:** Aparece **al inicio de cada página**. Normalmente incluye: Nombre de la empresa, fecha, identificación del informe ...
- **Column Header.** Se utiliza para mostrar los **encabezados de las columnas de datos**, especialmente en informes tabulares.
- **Detail.** Es la **banda más importante del informe**. Se repite una vez por cada registro de la fuente de datos. Aquí se muestran los valores reales de los campos.
- **Column Footer.** Permite mostrar información al final de cada columna, como subtotales.
- **Page Footer.** Se muestra al final de cada página. Es habitual incluir: Numeración de páginas, información legal o pie corporativo.
- **Summary.** Se **muestra una sola vez, al final del informe**. Se utiliza para totales generales los resúmenes.
- **No Data:** Se muestra cuando la fuente de datos no devuelve registros; informa al usuario que no hay datos disponibles.
- **Last Page Footer:** Aparece únicamente en la última página del informe; se usa para información final especial, como firmas o notas finales.



En Jaspersoft Studio, las bandas de un informe **no son obligatorias**.

El diseñador puede **añadirlas o eliminarlas según las necesidades del informe**.

Aunque algunas bandas, como **Detail**, suelen ser esenciales en informes con datos. La flexibilidad para agregar o quitar bandas permite adaptar el diseño del informe al tipo de información que se desea presentar.



4. Orígenes de datos

Un informe, por definición, **no tiene sentido sin datos**. El diseño del informe determina cómo se presentará la información, pero es el **origen de datos** el que aporta los valores reales que se mostrarán al usuario final. Por este motivo, la correcta definición y gestión de los orígenes de datos es uno de los aspectos más importantes en cualquier sistema de generación de informes.

JasperReports y Jaspersoft Studio destacan precisamente por su capacidad para trabajar con **múltiples tipos de fuentes de datos**, lo que permite utilizar la misma herramienta en contextos muy diversos: desde aplicaciones empresariales conectadas a bases de datos relacionales hasta sistemas que generan informes a partir de ficheros .csv o .json.

4.1 JDBC (MySQL / PostgreSQL / MariaDB)

JDBC como origen de datos principal

El uso de **bases de datos relacionales** es el escenario más habitual en aplicaciones Java profesionales, y por tanto, **JDBC** es el origen de datos más utilizado en JasperReports.

JDBC (Java Database Connectivity) es la API estándar de Java para conectarse a bases de datos relacionales. JasperReports se apoya directamente en JDBC para:

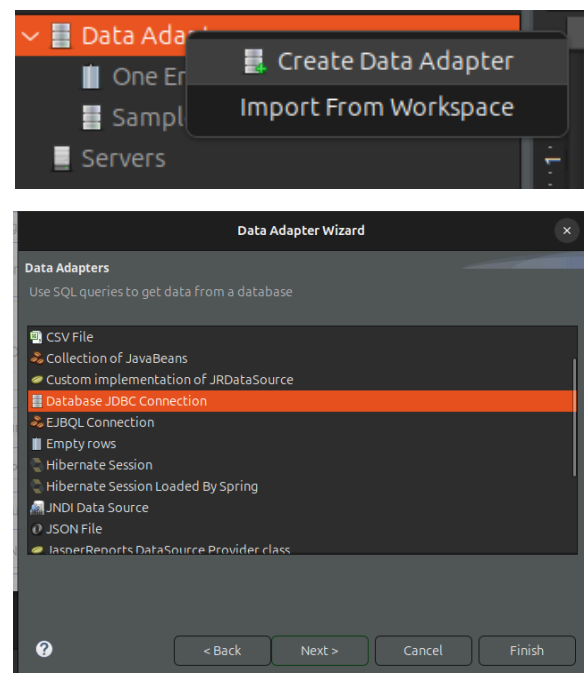
- Establecer la conexión con la base de datos.
- Ejecutar consultas SQL.
- Recuperar los resultados.
- Asociar las columnas devueltas con los campos del informe.

Funcionamiento en los informes

Cuando un informe utiliza JDBC como origen de datos, el flujo general es el siguiente:

1. El informe define una **consulta SQL**.
2. El motor JasperReports ejecuta dicha consulta usando una conexión JDBC.
3. Cada fila del resultado se convierte en un registro del informe.
4. Las columnas del **ResultSet** se asignan a los **campos (Fields)** del informe.
5. La banda **Detail** se repite una vez por cada fila obtenida.

Este modelo encaja perfectamente con la estructura tabular de los informes clásicos (listados, facturas, resúmenes, etc.).



4.2 CSV, XML y JSON

Informes a partir de ficheros estructurados

JasperReports permite trabajar con **ficheros estructurados** como origen de datos.

CSV (Comma-Separated Values)

Los ficheros CSV representan datos tabulares en formato texto. JasperReports permite:

- Definir el delimitador.
- Asociar columnas a campos del informe.
- Procesar grandes volúmenes de datos de forma sencilla.

XML

XML es un formato jerárquico ampliamente utilizado en integración de sistemas.

JasperReports permite:

- Definir una expresión XPath.
- Extraer nodos como registros.
- Mapear elementos XML a campos del informe.

JSON

JSON es hoy en día uno de los formatos más utilizados para el intercambio de datos, especialmente en APIs REST y JasperReports permite utilizar documentos JSON como origen de datos. En este caso, los campos del informe se asocian a claves o rutas dentro del documento JSON.

4.3 Uso de Data Adapters

Un **Data Adapter** es un objeto de configuración que describe **cómo acceder a un origen de datos concreto** desde Jaspersoft Studio.

El Data Adapter se utiliza **solo en tiempo de diseño y pruebas**. En producción, la aplicación Java proporciona la conexión real.

5. Creación de informes. Primeros pasos.

DATOS UTILIZADOS:

Dataset and Query Dialog

Sample DB

Query | Java Bean

Language | SQL

Read Fields

> INFORMATION_SCHEMA

> PUBLIC (CURRENT)

> SYSTEM_LOBS

DOCUMENT

☒ ID

☒ ADDRESSID

☒ TOTAL

☒ ID

☒ FIRSTNAME

☒ LASTNAME

☒ STREET

☒ CITY

Texts

Outline

Diagram

Field Name	Class Type	Description	Column name	Column label	Column index	Time zone	Properties
ID	java.lang.Integer						3 Properties
TOTAL	java.math.BigDe...						3 Properties
ADDRESSID	java.lang.Integer						3 Properties
COLUMN_4	java.lang.Integer						3 Properties
FIRSTNAME	java.lang.String						3 Properties
LASTNAME	java.lang.String						3 Properties
STREET	java.lang.String						3 Properties
CITY	java.lang.String						3 Properties

Fields

Parameters

Sorting

Filter Expression

Data preview

?

OK

Cancel

RESULTADO DEL INFORME:

NIF	12345678A	Fecha	23/1/26 10:29
Empresa	Chan do Monte		
Direccion	Direccion Inventada 12 , 36123		

VENTAS

ID	FIRSTNAME	LASTNAME	TOTAL
0	Laura	Steel	2607,60
18	Sylvia	Ringer	3772,80
35	Sylvia	Ringer	3102,60
40	Sylvia	Ringer	5288,40
39	Laura	Miller	1311,60
21	Andrew	Miller	3740,10
19	James	Schneider	804,30
30	Laura	Ringer	2117,40
43	Laura	Ringer	4827,60

6. Gestión de errores

Al generar la vista previa, Jaspersoft Studio realiza una serie de operaciones para crear el informe final.

La primera operación consiste en compilar el archivo fuente, con extensión `.jrxml`, en un archivo Jasper con extensión `.jasper`.

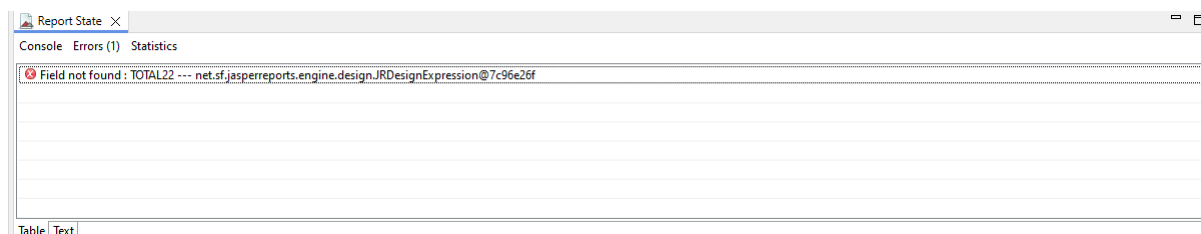
Este primer paso puede fallar si los elementos no están colocados correctamente (por ejemplo, si un elemento se sitúa fuera de una banda) o si alguna expresión del informe contiene errores y no puede ser compilada.

Si la compilación se realiza correctamente, el archivo `.jasper` generado se carga y se rellena utilizando la conexión activa o la fuente de datos definida.

Esta segunda operación también puede producir errores, por ejemplo, si la base de datos referenciada no está disponible, si se ha definido una consulta no válida o si una expresión genera un error debido a un valor nulo durante el proceso de llenado.

Finalmente, si todas las operaciones se completan sin errores, el informe se muestra en el visor integrado de Jaspersoft Studio.

Los errores se muestran en la ventana de Problemas, mientras que los resultados de la ejecución del informe aparecen en la salida de Jaspersoft Studio, ambas ubicadas en la parte inferior del entorno de trabajo.



7. Formatos de salida.

El objetivo de un informe es **presentar la información en un formato accesible y fácil de distribuir**. Por ello, una vez generado, el informe se convierte en archivos que pueden almacenarse, compartirse e imprimirse.

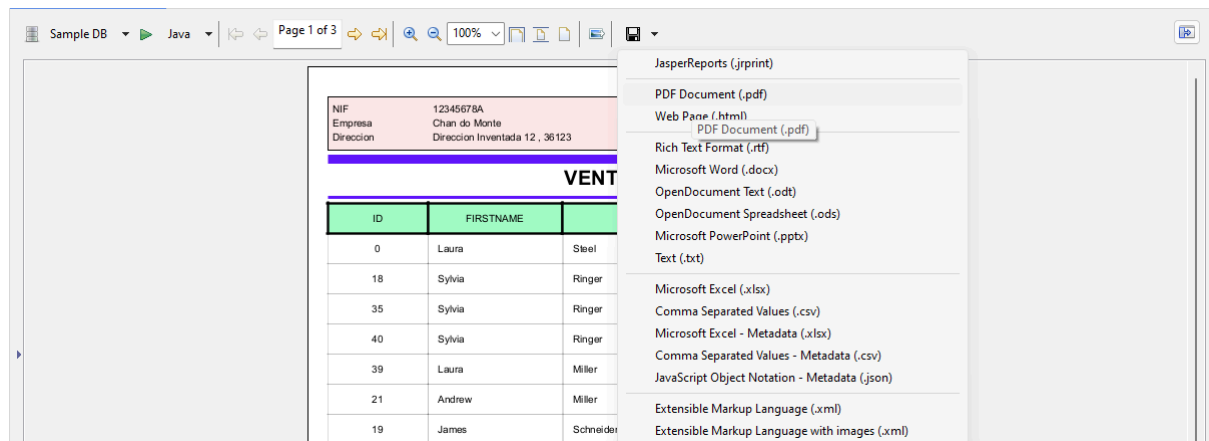
Cualquier motor de informes debe ser capaz de **exportar sus documentos a formatos de uso habitual**, como **PDF, HTML, Excel**, entre otros.

En **Jaspersoft Studio**, esta funcionalidad se gestiona desde la opción **Vista previa (Preview)**, donde es posible **seleccionar el formato final del informe** antes de su visualización o exportación.

Al desplegar el menú de vista previa, se muestran los **distintos formatos de salida disponibles**, aplicados sobre la previsualización del informe.

Cuando se establece un formato de vista previa, el informe se **exporta automáticamente al formato seleccionado** al ejecutar la vista previa, y se abre la **aplicación de visualización correspondiente**, como un visor de PDF o una hoja de cálculo.

Otra opción es el icono de exportar (disquetera) y permite exportarlo a los diferentes formatos.



8. Definir parámetros de entrada en Jaspersoft Studio

Los **parámetros de entrada** permiten que un informe reciba valores externos en el momento de su ejecución. Estos valores pueden ser introducidos por el usuario, enviados desde la aplicación Java o definidos dinámicamente, y se utilizan para **filtrar datos**, **modificar el comportamiento del informe** o **personalizar el contenido mostrado**.

Desde el punto de vista arquitectónico, los parámetros actúan como el **punto de conexión entre el informe y el exterior**, evitando que el informe tenga valores fijos o dependientes del código SQL.

8.1 Qué es un parámetro

Un parámetro es una variable definida en el informe que:

- Tiene un **nombre**
- Un **tipo de dato Java**
- Puede (o no) solicitarse al usuario
- Se evalúa **antes de ejecutar la consulta**

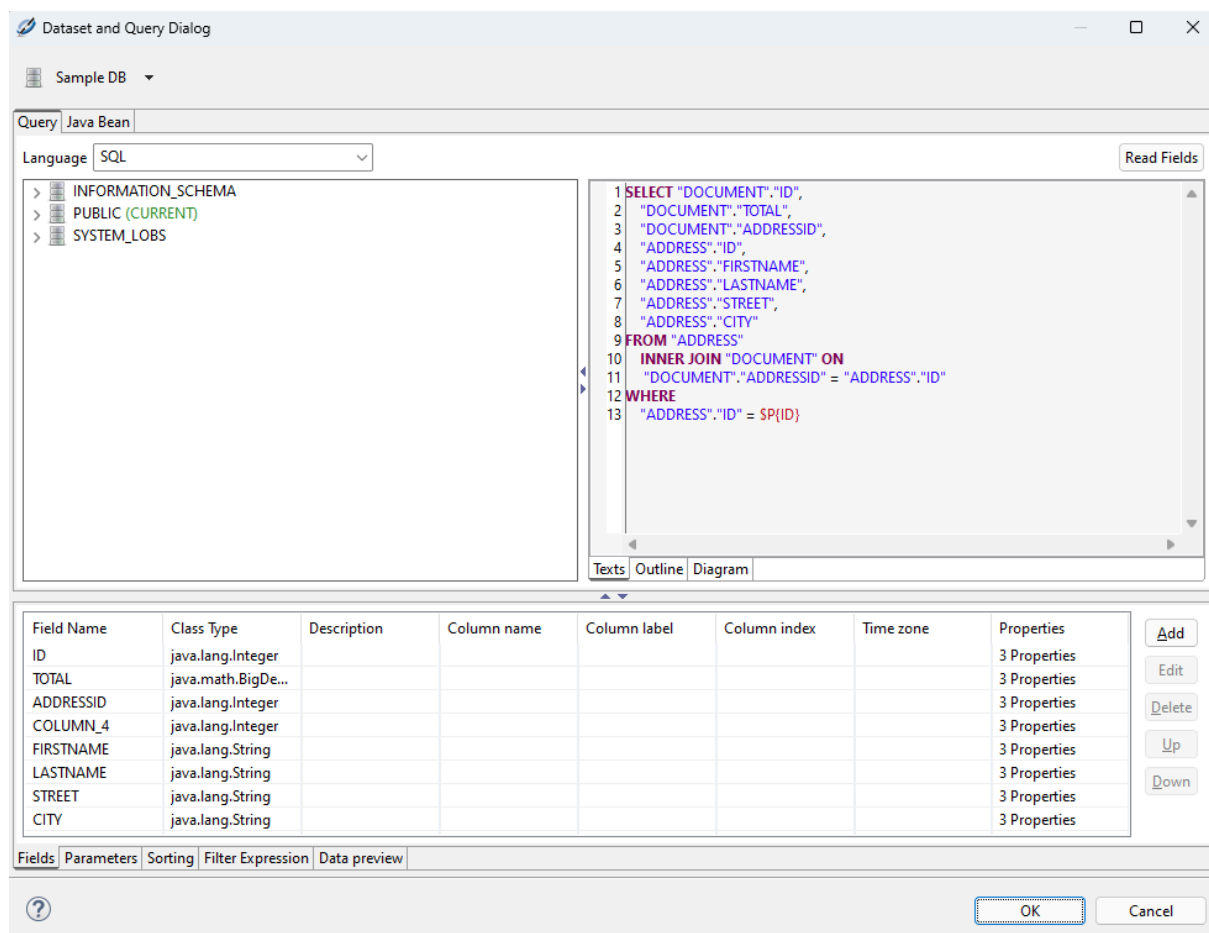
En Jaspersoft Studio, los parámetros se definen desde el **Outline** o desde el panel **Parameters** del informe.

Para crear un parámetro:

1. Abrir el informe en modo **Design**
2. En el panel **Outline**, botón derecho sobre *Parameters*
3. Seleccionar **Create Parameter**
4. Definir:
 - **Name:** por ejemplo **ID**
 - **Class:** `java.lang.Integer`
 - **Is For Prompting:** `true` (para que lo pida en Preview)
 - **Default Value Expression:** vacío (si no queremos valor fijo)

Este parámetro quedará disponible para todo el informe.

8.3 Uso del parámetro en la consulta SQL



En la imagen adjunta se observa el **Dataset and Query Dialog**, donde se define una consulta SQL con un parámetro:

Explicación del uso del parámetro

- `$P{ID}` hace referencia al **parámetro del informe llamado ID**
- Antes de ejecutar la consulta:
 1. JasperReports solicita el valor del parámetro (en Preview o desde Java)
 2. Sustituye `$P{ID}` por el valor introducido
 3. Ejecuta la consulta ya filtrada

De este modo:

- El informe **no devuelve todos los registros**
- Solo se obtienen los datos correspondientes a la dirección cuyo ID coincide con el valor del parámetro

8.5 Uso del parámetro dentro del informe

Además de usarse en la consulta, un parámetro puede mostrarse o utilizarse dentro del informe. Arrastrando el parámetro al informe se introduce el parámetro ejemplo: `$P{ID}`
Ejemplo : Cambiar títulos dinámicamente: "Factura nº " + `$P{ID}`

Esto permite que el informe sea **dinámico y reutilizable**, sin duplicar diseños.

En producción, el usuario **no interactúa con Jaspersoft Studio**. La aplicación Java es la que envía los parámetros al informe.

Conceptualmente:

- La aplicación crea un `Map<String, Object>`
- Inserta los valores de los parámetros
- Ejecuta el informe

8.7 Ventajas del uso de parámetros

El uso de parámetros aporta:

- Reutilización de informes
- Mayor seguridad (evita SQL dinámico peligroso)
- Flexibilidad
- Separación clara entre datos y presentación
- Informes más limpios y mantenibles

Un mismo informe puede servir para **cientos o miles de ejecuciones distintas** simplemente cambiando el valor de sus parámetros.

9. IMÁGENES

Las imágenes son representaciones visuales de objetos almacenados en un archivo con un formato específico.

Al añadir imágenes a un informe, no se incorporan directamente al informe, sino que se utiliza una expresión con la ruta del archivo de la imagen.

Por ello, al arrastrar un elemento de imagen en el diseñador, Jaspersoft Studio muestra un cuadro de diálogo para seleccionar el archivo. Esta es la forma más cómoda de indicar qué imagen se utilizará en el informe.

10. Frames

Qué es un Frame

Un *Frame* es un **contenedor** que permite agrupar varios elementos del informe (textos, campos, imágenes, líneas...) y tratarlos como una **unidad** dentro de una banda.

Para qué se utiliza

- Agrupar información relacionada (cabeceras, datos de cliente, totales...).
- Aplicar **bordes y fondo** comunes.
- Mover y alinear bloques completos fácilmente.
- Mejorar la **maquetación** y el orden del informe.

Cómo se crea y usa

- Se arrastra desde la **Palette** a una banda.
- Los elementos se colocan **dentro** del Frame y se posicionan de forma relativa.
- En **Outline** se ve la jerarquía Frame → elementos.

Propiedades clave

- Bordes (Box).
- Fondo (Opaque / color).
- Tamaño y posición.

Reutilización (Composite Element)

- Un Frame puede convertirse en **Composite Element**.
- Permite **reutilizar el bloque** en el mismo informe o en otros informes.
- Muy útil para cabeceras, pies y cajas estándar.

11. DATASET

Dataset y Subdataset en Jaspersoft Studio

En Jaspersoft Studio, los **datasets** son el mecanismo que define **qué datos se van a utilizar y cómo se obtienen** dentro de un informe. Aunque en informes sencillos este concepto pasa casi desapercibido, entender bien la diferencia entre *dataset principal* y *subdataset* es clave para trabajar con informes reales que incluyen tablas, gráficas, agrupaciones o resúmenes.

Dataset principal (Main Dataset)

Todo informe tiene un **dataset principal**, incluso aunque el diseñador no lo cree explícitamente. Este dataset es el que está asociado directamente al informe y se define desde el apartado **Dataset and Query**.

El dataset principal incluye:

- La **consulta** (normalmente SQL).
- Los **campos (Fields)** que provienen de esa consulta.
- Los **parámetros y variables** asociados.

Cuando el informe se ejecuta:

1. JasperReports ejecuta la consulta del dataset principal.
2. Cada fila devuelta se convierte en un registro del informe.
3. La banda **Detail** se repite una vez por cada registro.

Por este motivo, el dataset principal es el que se utiliza en la mayoría de los informes tabulares clásicos: listados, facturas, informes de clientes, etc. Todo lo que se arrastra directamente al Detail (campos `$F{...}`) pertenece siempre al dataset principal.

Subdataset: qué es y por qué existe

Un **subdataset** es un dataset adicional e independiente del principal. Tiene su **propia consulta**, sus propios campos y sus propios parámetros, y no depende automáticamente del flujo del informe.

La clave: **Un subdataset no se ejecuta por sí solo**, sólo se ejecuta cuando un elemento del informe lo necesita.

Los subdatasets existen porque hay elementos del informe que **no pueden trabajar directamente con el dataset principal**, o necesitan datos con una estructura distinta.

Cuándo se usan subdatasets

En Jaspersoft Studio, los subdatasets se utilizan principalmente en:

- **Tablas (Table)**
- **Gráficas (Chart)**
- **Crosstabs**
- **Lists**

Estos elementos necesitan su propio dataset porque:

- Pueden requerir **agregaciones** (SUM, COUNT, AVG...).
- Pueden necesitar **filtros distintos** a los del informe principal.
- Pueden mostrar datos que no encajan en la repetición del Detail.

Relación entre dataset principal y subdataset

Aunque son independientes, el dataset principal y los subdatasets **pueden comunicarse mediante parámetros**.

Un caso muy habitual:

- El dataset principal recorre clientes.
- Dentro del Detail hay una tabla con un subdataset.
- El subdataset recibe como parámetro el `$F{ID_CLIENTE}` del registro actual.
- La tabla muestra solo los datos relacionados con ese cliente.

De este modo, el subdataset se ejecuta **una vez por cada registro** del dataset principal, pero con su propia consulta y lógica.

Ventajas de trabajar con subdatasets

El uso de subdatasets aporta varias ventajas importantes:

- Permite **separar claramente** las consultas del informe.
- Evita consultas SQL excesivamente complejas en un único dataset.
- Facilita la creación de informes con **resúmenes, gráficos y vistas agregadas**.
- Hace el informe más **mantenible y comprensible**.
- Permite reutilizar el mismo subdataset en distintos elementos.

Desde un punto de vista profesional, el uso correcto de subdatasets es una señal clara de un diseño de informes bien estructurado.

12. AGRUPAMIENTOS.

Los **agrupamientos (Groups)** permiten **organizar y estructurar los datos** de un informe en función de un criterio común. En lugar de mostrar una lista plana de registros, los agrupamientos hacen posible dividir la información en bloques lógicos, facilitando la lectura, el análisis y la obtención de resúmenes parciales.

En informes reales, los agrupamientos son fundamentales para transformar datos en información útil.

Técnicamente, un grupo se define mediante:

- Un **nombre de grupo**.
- Una **expresión de agrupación** (normalmente un campo `$F{...}`).
- Dos bandas asociadas: **Group Header & Group Footer**.

Para qué se utilizan los agrupamientos

Los agrupamientos se utilizan para:

- Organizar registros relacionados (por cliente, por ciudad, por fecha, por categoría...).
- Añadir **encabezados** que identifiquen cada bloque de datos.
- Calcular **subtotales** y resúmenes parciales.
- Mejorar la legibilidad de informes largos.
- Preparar el informe para análisis visual y toma de decisiones.

Bandas de un agrupamiento

Cada grupo crea dos bandas específicas:

Group Header

Se muestra al inicio de cada grupo. Se utiliza para:

- Mostrar el valor del campo de agrupación (por ejemplo, nombre del cliente).
- Incluir títulos de sección.
- Separar visualmente bloques de datos.

Group Footer

Se muestra al final de cada grupo. Se utiliza para:

- Mostrar subtotales, contadores o medias.
- Añadir separadores visuales entre grupos.

Estas bandas permiten insertar información contextual sin interferir con la banda Detail.

En JasperReports / Jaspersoft Studio, para que un agrupamiento funcione correctamente, los datos **DEBEN venir ordenados por el mismo campo (o expresión) que se usa para agrupar**.

LISTADO DE CONTACTOS

ID	FIRSTNAME	LASTNAME	STREET	CITY
9	James	Schneider	277 Seventh Av.	Berne
22	Bill	Ott	250 - 20th Ave.	Berne
23	Julia	Heiniger	358 College Av.	Boston
32	Michael	Ott	339 College Av.	Boston
11	Julia	White	412 Upland Pl.	Chicago
35	George	Karsen	412 College Av.	Chicago
39	Mary	Karsen	202 College Av.	Chicago
0	Laura	Steel	429 Seventh Av.	Dallas
4	Sylvia	Ringer	365 College Av.	Dallas
10	Anne	Fuller	135 Upland Pl.	Dallas
19	Susanne	Heiniger	86 - 20th Ave.	Dallas
36	John	Steel	276 Upland Pl.	Dallas
37	Michael	Clancy	19 Seventh Av.	Dallas
40	Susanne	Miller	440 - 20th Ave.	Dallas

LISTADO DE CONTACTOS

ID	FIRSTNAME	LASTNAME	STREET	CITY
Berne				
9	James	Schneider	277 Seventh Av.	Berne
22	Bill	Ott	250 - 20th Ave.	Berne
Número de registros de la ciudad de Berne es de 2 registros.				
Boston				
23	Julia	Heiniger	358 College Av.	Boston
32	Michael	Ott	339 College Av.	Boston
Número de registros de la ciudad de Boston es de 2 registros.				
Chicago				
11	Julia	White	412 Upland Pl.	Chicago
35	George	Karsen	412 College Av.	Chicago
39	Mary	Karsen	202 College Av.	Chicago
Número de registros de la ciudad de Chicago es de 3 registros.				
Dallas				
0	Laura	Steel	429 Seventh Av.	Dallas

13. TABLAS.

Las **tablas (Tables)** son uno de los componentes muy utilizado en Jaspersoft Studio para representar información estructurada de forma clara y ordenada. Pensado para mostrar **conjuntos de datos autocontenidos** dentro de cualquier parte del informe.

Qué es una tabla

Una tabla es un **componente compuesto** que muestra datos en filas y columnas, donde cada columna está asociada a un campo del dataset que la alimenta. A diferencia del Detail, la tabla:

- No depende directamente del dataset principal.
- Utiliza siempre un **subdataset** propio.
- Se puede insertar en cualquier banda del informe (Title, Page Header, Detail, Group Header, Summary, etc.).

Esto convierte a las tablas en una herramienta ideal para mostrar **resúmenes, listados secundarios o datos relacionados** con el contexto del informe.

Relación entre tablas y subdatasets

Toda tabla en Jaspersoft Studio está asociada obligatoriamente a un **subdataset**.

Este subdataset define:

- La consulta que obtiene los datos de la tabla.
- Los campos disponibles para las columnas.
- Los parámetros que permiten filtrar o personalizar los datos.

Cuando el informe se ejecuta el subdataset de la tabla se ejecuta **solo cuando la tabla se va a mostrar**.

Estructura interna de una tabla

Una tabla está formada internamente por varias secciones, similares a las bandas de un informe:

- **Table Header:** se muestra una sola vez al inicio de la tabla.
- **Column Header:** contiene los títulos de las columnas.
- **Detail:** se repite una vez por cada registro del subdataset.
- **Column Footer:** se utiliza para subtotales por columna.
- **Table Footer:** se muestra al final de la tabla completa.

Esta estructura permite controlar con precisión tanto la presentación de los datos como los cálculos asociados a cada columna.

Creación de una tabla (flujo general)

El proceso típico para crear una tabla es el siguiente:

1. Arrastrar el componente **Table** desde la Palette al informe.
2. Seleccionar o crear un **subdataset** para la tabla.
3. Definir la consulta del subdataset.
4. Añadir columnas y asociarlas a campos del subdataset.
5. Ajustar el diseño: anchos, alineación, formatos y estilos.
6. Previsualizar y corregir posibles errores.

Cuándo usar una tabla y cuándo no

Usa tablas cuando:

- Necesitas mostrar un conjunto de datos **independiente** del dataset principal.
- Quieres incluir un listado secundario dentro de un informe.
- Trabajas con resúmenes o datos relacionados.

No son necesarias cuando:

- El informe es un listado simple que puede resolverse solo con la banda Detail.
- No se requiere una estructura tabular compleja.

The screenshot shows the 'Table Wizard' dialog box with the 'Layout' tab selected. The dialog is titled 'Table Wizard' and has a close button (X) in the top right corner. The main heading is 'Layout' with the instruction 'Please select layout'. The dialog is divided into several sections: 'Cell Colors' with checkboxes for 'Table Header', 'Column Header', 'Detail', and 'Alt. Row Detail', and an option for 'Use alternated detail rows background'; 'Colors by Schema' with a 'Color scheme' dropdown set to 'Aliceblue' and a 'Variations' dropdown set to 'Default'; 'Cell borders' with 'Borders color' set to '#000000' and 'Borders style' set to a grid pattern. A 'Style preview' section on the right shows a 6x3 grid with alternating light blue and white rows. At the bottom, there are six checkboxes, all of which are checked: 'Add Table Header', 'Add Column Header', 'Add Group Header', 'Add Group Footer', 'Add Column Footer', and 'Add Table Footer'. The bottom of the dialog features a help icon (question mark), a '< Back' button, a 'Next >' button, a 'Cancel' button, and a green 'Finish' button.

Table Wizard

Layout
Please select layout

Cell Colors

Table Header ☐ Column Header ☐

Detail ☐ Alt. Row Detail ☐

☐ Use alternated detail rows background

Colors by Schema

Color scheme

Variations

Cell borders

Borders color Borders style

Style preview

☒ Add Table Header ☒ Add Column Header ☒ Add Group Header

☒ Add Group Footer ☒ Add Column Footer ☒ Add Table Footer

14. Gráficos

Los gráficos (Charts) permiten representar la información de forma visual, facilitando la interpretación de datos, la detección de tendencias y la comparación entre valores. En informes profesionales, los gráficos no sustituyen a las tablas o listados, sino que los complementan, aportando una visión más inmediata y analítica de la información.

En Jaspersoft Studio, los gráficos forman parte de los componentes avanzados del informe y están pensados para mostrar datos agregados o resumidos.

Qué es un gráfico

Un gráfico es un componente visual que representa datos mediante elementos gráficos como barras, líneas, sectores o áreas. Cada gráfico se construye a partir de:

- Un tipo de gráfico (barras, líneas, sectores, etc.).
- Un subdataset, que proporciona los datos.
- Una serie de expresiones que indican qué valores se representan.

Al igual que las tablas, los gráficos:

- No utilizan directamente el dataset principal.
- Siempre trabajan con un subdataset propio.
- Pueden insertarse en cualquier banda del informe.

Tipos de gráficos más habituales

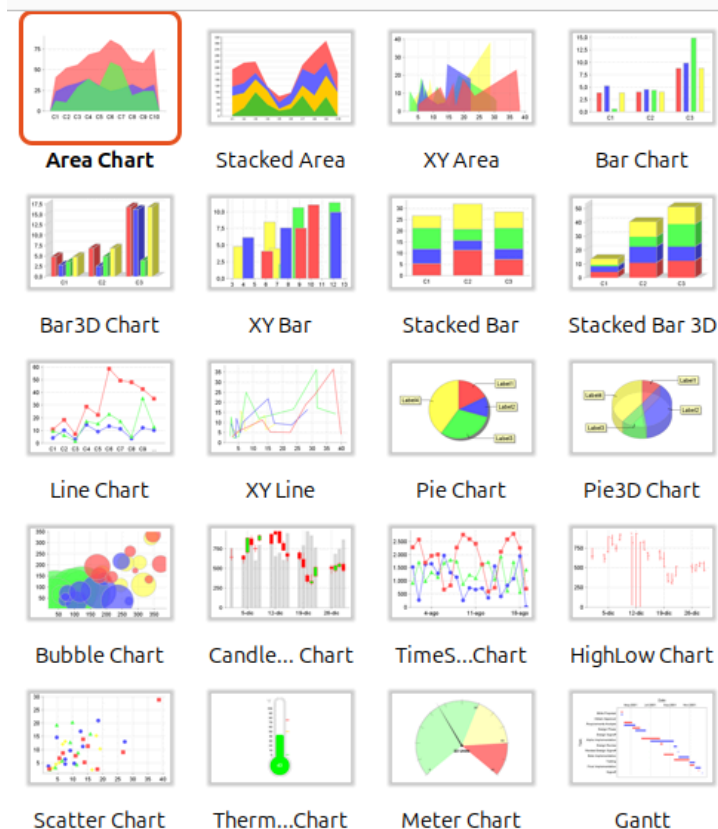
Jaspersoft Studio ofrece múltiples tipos de gráficos, entre los más comunes:

- **Bar Chart:** comparaciones entre categorías.
- **Line Chart:** evolución temporal o tendencias.
- **Pie Chart:** distribución porcentual.
- **Area Chart:** evolución acumulada.
- **Stacked Charts:** comparaciones acumuladas.

La elección del tipo de gráfico depende del mensaje que se quiera transmitir y del tipo de datos disponibles.

Los gráficos son especialmente útiles cuando:

- Se quiere mostrar tendencias o evoluciones.
- Se comparan valores entre categorías.
- Se desea ofrecer una visión rápida del estado de los datos



15. INTEGRACIÓN CON JAVA

